Case Study Report

SAFE SAFAR

Multi-Agent Air Traffic Collision Avoidance and Dynamic Rerouting System

Submitted By

Aditya Reddy, CB.SC.U4CSE23707

Harish Veylan, CB.SC.U4CSE23722

Konduru Hemesh, CB.SC.U4CSE23724

Rithvik Mukka, CB.SC.U4CSE23731

in partial fulfilment of the requirements for the course of

23CSE401 - Fundamentals of Artificial Intelligence

Academic Year 2026-27 Odd Semester



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

AMRITA SCHOOL OF COMPUTING

AMRITA VISHWA VIDYAPEETHAM

COIMBATORE - 641 112

Contents

1 Introduction **2**

1.1 Problem Statement 2

1.2 Objectives 2

1.3 Scope 2

2 PEAS Description **2**

2.1 Performance Measure 3

2.2 Sensor and Actuator Mappings 3

3 Agent and Environment Analysis **4**

3.1 Environment Classification 4

3.2 Agent Category 4

4 Design **4**

4.1 State-Space Model 4

4.2 A* Search with Euclidean Heuristic 5

4.3 Dynamic Conflict Penalties and Rerouting 6

4.4 Sensing and Trajectory Prediction 6

4.5 Priority Negotiation and Conflict Clustering 7

4.6 Hardware and Software Requirements 8

4.7 System Architecture 9

5 Implementation **9**

5.1 Input Representation 9

5.2 Project Structure 9

5.3 Modules and Core Functions 10

5.4 Execution Flow 11

5.5 User Interface 11

6 Testing and Results **11**

6.1 Test Cases 11

6.2 Live Demonstration Procedure 12

6.3 Simulation Screenshots 13

6.4 Limitations 13

7 Conclusion and Future Work **13**

1 Introduction

SAFE SAFAR is a browser-based, multi-agent simulation of air traffic collision avoidance and dynamic rerouting. Each aircraft is modelled as an autonomous agent that senses nearby traffic, predicts future positions, negotiates right-of-way with the other agents involved in a conflict, and replans its route with A* search. When no acceptable detour exists, the agent falls back to a simulated holding pattern.

1.1 Problem Statement

When aircraft fly on intersecting routes, their predicted trajectories can enter conflict. Several conflicts may occur at once and affect each other, because one aircraft's reroute changes the conflict picture for the others. Each aircraft must also decide locally, from limited perception, rather than relying on a single omniscient controller.

1.2 Objectives

1. Detect predicted conflicts early.
2. Settle right-of-way between conflicting aircraft by priority.
3. Reroute the yielding aircraft with A* search.
4. Hold when no acceptable detour exists.
5. Keep decisions deterministic, so every agent reaches the same answer.

1.3 Scope

The system is an academic prototype that runs in a web browser. It uses a 2D waypoint graph with simulated sensing and aircraft dynamics. It is *not* certified air traffic control software and must not be used as aviation guidance.

2 PEAS Description

The task environment is specified through the PEAS framework (Table 1).

Component	Description
Performance	Avoid predicted conflicts; reach the destination; minimise distance, fuel and delay; reduce deadlocks.
Environment	2D waypoint graph of 77 fixes in an 11×7 grid; moving aircraft; airway segments; conflict zones.
Actuators	Heading changes and waypoint updates; A* rerouting and speed adjustment; holding-pattern behaviour.
Sensors	Own position, heading, speed and fuel; destination; information on nearby aircraft within the sensing range.

Table 1: PEAS description of SAFE SAFAR.

2.1 Performance Measure

Decisions are guided by a weighted cost

$$\text{Cost} = \lambda \cdot \text{CollisionRisk} + w_d \cdot \text{Distance} + w_f \cdot \text{Fuel} + w_t \cdot \text{Delay}. \tag{1}$$

The weight λ on collision risk is intended to dominate, while w_d, w_f and w_t trade off distance, fuel and delay. A weighted cost guides decisions but does not by itself guarantee collision avoidance. The concrete success indicators are (i) predicted separation staying above the configured conflict threshold of 34 NM, (ii) every aircraft reaching its destination, and (iii) absence of circular-yielding deadlocks.

2.2 Sensor and Actuator Mappings

Input (sensor)	Output (actuator)
Own position, heading, speed, fuel $\rightarrow$ agent state	Waypoint updates and heading changes along the queued route
Destination $\rightarrow$ goal for A*	Route replacement after A* rerouting
Nearby aircraft within 165 NM and the 72° radar cone $\rightarrow$ predicted conflicts	Yield, speed adjustment, or entry into a holding orbit

Table 2: Input–sensor and output–actuator mapping.

3 Agent and Environment Analysis

3.1 Environment Classification

Property	In SAFE SAFAR
Multi-agent	Several autonomous aircraft act at the same time.
Dynamic	Traffic keeps changing while plans are being made.
Partially observable	Sensing is limited by range and field of view.
Sequential	Each decision shapes later states.
Uncertain (stochastic)	Prediction error and the other agents' choices introduce uncertainty.
Cooperative	The safety goal is shared; route preferences are individual.

Table 3: Classification of the task environment.

3.2 Agent Category

Each aircraft is a **hybrid agent**. Its *reactive* part responds to predicted conflicts, and its *goal-based* part plans a route to the destination. The agent repeats the loop

$$\text{Observe} \rightarrow \text{Predict} \rightarrow \text{Negotiate} \rightarrow \text{Act} \rightarrow \text{Replan} \rightarrow \text{Observe again.}$$

The agents are *coupled*: when one aircraft commits to a new route, its predicted trajectory changes what its neighbours sense. Agents therefore re-predict on every cycle instead of trusting an old plan.

4 Design

4.1 State-Space Model

The airspace is a graph $G = (V, E)$. V holds the 77 waypoint nodes of an 11×7 grid and E the connected airway segments. The base edge cost is the segment length, with dynamic penalties added when conflicts are predicted. An aircraft's state consists of its position, heading, speed, fuel, destination and queued route. Route planning is a search of G for a low-cost path from the aircraft's current waypoint to its destination; as costs change with predicted conflicts, the path is recomputed when needed.

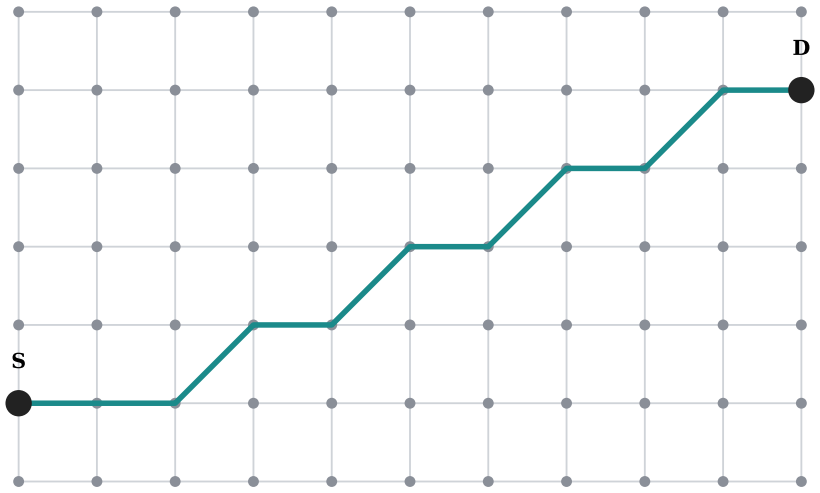


Figure 1: Schematic of the 11×7 waypoint graph. S is the start, D the destination and the highlighted path is a candidate route.

4.2 A* Search with Euclidean Heuristic

Routes are found with A*, which expands the node with the lowest

$$f(n) = g(n) + h(n), \tag{2}$$

where $g(n)$ is the accumulated route cost and $h(n)$ the Euclidean distance to the destination. When edge costs are geometric lengths, straight-line distance never overestimates the remaining cost, so h is admissible (and consistent). On a finite graph with non-negative costs, A* then returns a least-cost route.

Table 4 traces the illustrative example of Figure 2 (coordinates are illustrative).

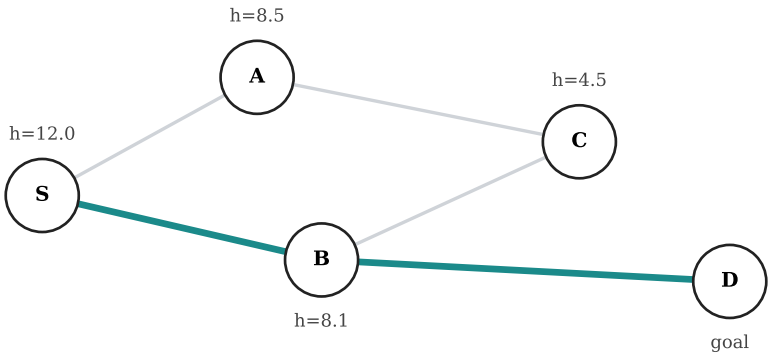


Figure 2: Illustrative A* example; the highlighted path S→B→D is the route found.

Step	Expand	Open set (f)
1	S ($f = 12.0$)	A 13.5, B 12.2
2	B ($f = 12.2$)	D 12.2, A 13.5, C 13.6
3	D = goal	Path S→B→D, $g = 12.2$

Table 4: A* trace for the illustrative example. A and C are never expanded.

4.3 Dynamic Conflict Penalties and Rerouting

To steer routes away from predicted conflicts, A* uses a penalised evaluation function

$$f'(n) = g(n) + h(n) + \text{Penalty}(n), \tag{3}$$

$$\text{Penalty}(n) = \sum_k \max\big(0,\ (95 - d_k) \times 14\big), \tag{4}$$

where d_k is the distance from a candidate node to the k-th predicted conflict location. The values 95 and 14 are configured parameters. Nodes within 95 units of a conflict therefore pay a cost that grows towards it, and A* prefers other routes. If the best alternative adds more than 35 NM, or no alternative exists, the agent uses the simulated holding pattern. Penalties steer routes; they do not guarantee collision avoidance.

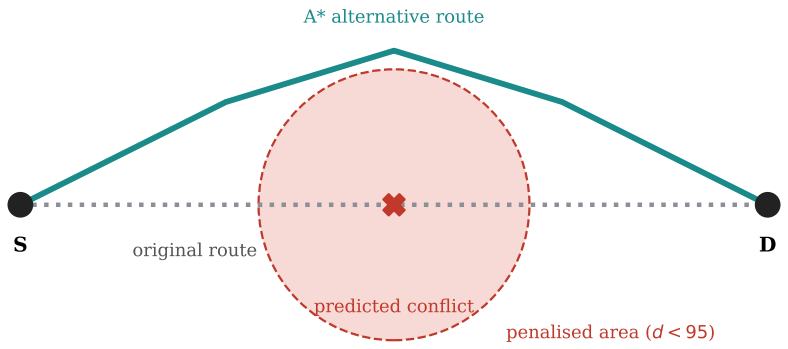


Figure 3: A predicted conflict raises the cost of nearby nodes, so the yielding aircraft's A* search selects a detour.

4.4 Sensing and Trajectory Prediction

Each agent senses locally: another aircraft is considered only if it lies within the forward sensing horizon and the radar field of view. Agents estimate future relative positions at fixed checkpoints and flag a potential conflict when predicted separation falls below the configured threshold.

Parameter	Value
Forward sensing horizon	165 NM
Radar field of view	72°
Prediction checkpoints	0, 2, 4, 6, 8 s
Conflict threshold (configured)	34 NM
Conflict-resolution interval (where implemented)	≈350 ms

Table 5: Sensing and prediction parameters.

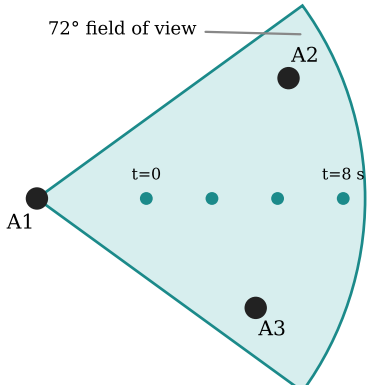


Figure 4: Schematic radar cone of aircraft A1 with projected positions at the prediction checkpoints (cone length is schematic).

4.5 Priority Negotiation and Conflict Clustering

When a conflict is predicted, right-of-way is decided by a priority score

$$P_i = \left(1 - \frac{F_i}{F_{\max,i}}\right) \times 60 + 0.4\, w_{p,i}, \tag{5}$$

where F_i is the remaining fuel, $F_{\max,i}$ the reference maximum and $w_{p,i}$ the mission-priority weight. The fuel term gives a higher score to an aircraft with less fuel. For two aircraft i and j and a hysteresis margin $\varepsilon = 0.6$:

- if $|P_i - P_j| > \varepsilon$, the aircraft with the higher score keeps the right-of-way;
- otherwise, a deterministic aircraft-ID tie-break is used, so every agent reaches the same answer.

Where three or more aircraft are mutually in conflict, a Union-Find structure groups them into a cluster, and routes are committed one after another in priority order, which avoids circular waiting.

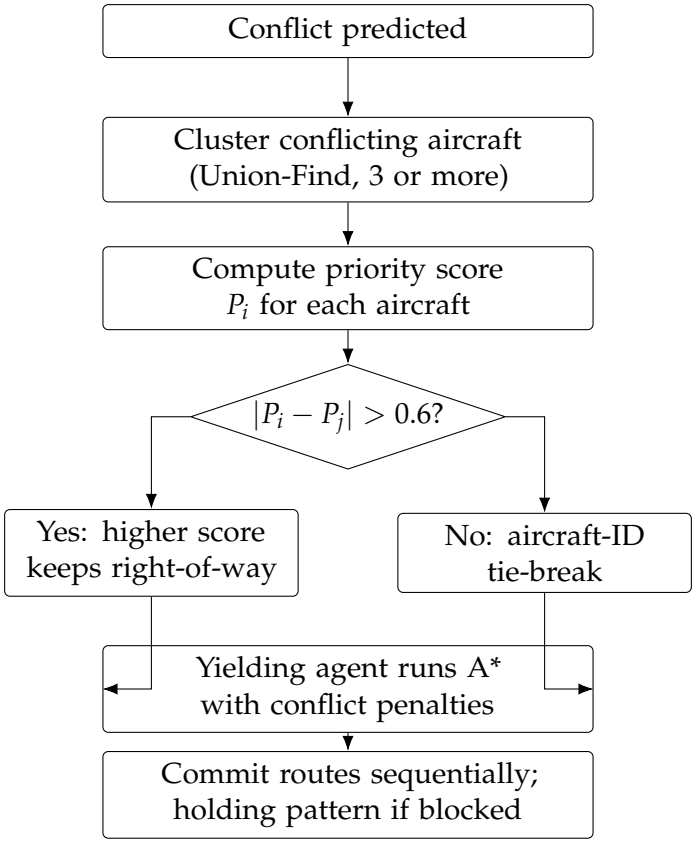


Figure 5: Priority negotiation and conflict clustering workflow.

4.6 Hardware and Software Requirements

SAFE SAFAR is built only from browser-native tools; no frameworks, machine-learning models, cloud services or external packages are used.

Tool	**Role**
JavaScript (ES6+)	Simulation logic: agents, sensing, A*, negotiation
HTML5 Canvas 2D	Draws the airspace, aircraft, routes and conflicts
CSS3	Interface layout and controls
performance.now()	Timing measurements
Modern web browser	Execution environment

Table 6: Tools and their roles.

The browser-native approach needs minimal setup, gives direct visual feedback, is portable, and keeps the simulation logic separate from rendering. No special hardware is needed beyond a computer able to run a modern web browser.

4.7 System Architecture

The architecture has five layers: (1) airspace topology (waypoint graph and airways); (2) multi-agent kinematics (aircraft state and motion); (3) sensing and clustering (radar cone, prediction, Union-Find); (4) search and decision-making (priority, A*, holding); and (5) visualisation (Canvas 2D rendering).

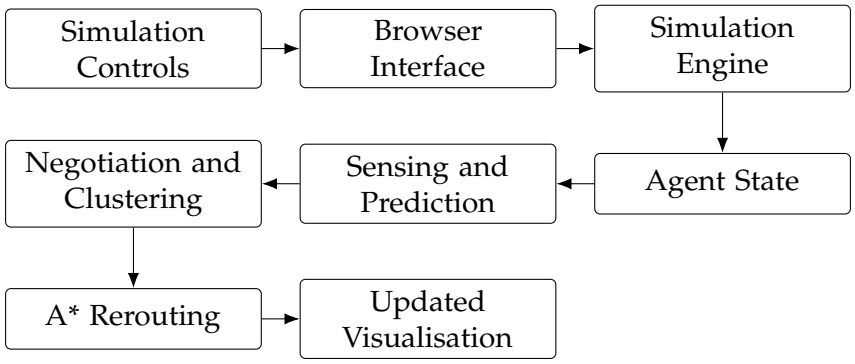


Figure 6: Software architecture and data flow.

Scalability (qualitative). Local sensing limits which aircraft pairs are compared, and clustering groups only the aircraft that interact. Actual cost depends on the implementation and the number of active conflicts; no large-scale benchmark was carried out.

5 Implementation

5.1 Input Representation

The airspace is represented as the 11×7 waypoint grid (77 nodes joined by edges). Each aircraft is represented by its state (position, heading, speed, fuel, destination) together with a queue of waypoints forming its route. No external dataset is used; traffic is generated by the simulation.

5.2 Project Structure

```
Safe-Safar--Multi-agent-air-traffic-collision-and-avoidance-system/
  .gitignore
  README.md
  index.html
  src/
    css/simulation.css
    js/simulation.js
```

File	Responsibility
`.gitignore`	Files and folders excluded from version control.
`README.md`	Project overview, architecture, mathematical models, PEAS formulation and usage instructions.
`index.html`	Main entry point: interactive simulation interface and case-study content; links the CSS and JavaScript.
`simulation.css`	Avionics-themed layout, HUD, aircraft panels, drawer, controls and radar-style visual effects.
`simulation.js`	Core engine: aircraft movement, A* pathfinding, trajectory prediction, conflict detection, priority negotiation, rerouting and Canvas rendering.

Table 7: File responsibilities.

5.3 Modules and Core Functions

Table 8 lists the main functions of `simulation.js`.

Function / module	Purpose
`nodeIndex(), graph init`	Represents the airspace as an 11×7 grid of 77 waypoint nodes joined by edges.
`astar()`	Finds a low-cost route using accumulated cost, the Euclidean heuristic and supplied penalties.
`predictPosition(ac, tSec)`	Estimates an aircraft's future position along its queued route.
`canSenseEachOther()`	Checks the sensing-range and radar-cone conditions between two aircraft.
`detectConflicts()`	Compares predicted separation at 0, 2, 4, 6, 8 s with the safety threshold.
`find(), union(), clusterConflicts()`	Group connected aircraft conflicts using the Union-Find technique.
`priorityScore()`	Ranks aircraft by remaining-fuel urgency and mission-priority weight.
`tryReroute()`	Runs A* with conflict-point penalties; validates routes and rejects excessive detours.
`resolveConflicts()`	Orders aircraft by priority and commits routes in turn; holds if rerouting fails.
`step(), frame()`	Update movement and fuel, run periodic conflict checks and drive the animation loop.
`render() and draw functions`	Draw the waypoint graph, aircraft, routes, predicted conflicts and visual effects on the Canvas.

Table 8: Core functions in `simulation.js`.

5.4 Execution Flow

Each simulation cycle follows nine stages:

1. **Initialise airspace** – `nodeIndex()`, graph initialisation.
2. **Spawn aircraft** – aircraft state and queued routes.
3. **Predict trajectories** – `predictPosition()`.
4. **Detect conflicts** – `canSenseEachOther()`, `detectConflicts()`.
5. **Cluster conflicts** – `find()`, `union()`, `clusterConflicts()`.
6. **Rank aircraft** – `priorityScore()`.
7. **Reroute with A*** – `tryReroute()`, `astar()`.
8. **Hold if blocked** – `resolveConflicts()` fallback.
9. **Update and render** – `step()`, `frame()`, `render()`.

In short, `simulation.js` combines pathfinding, multi-aircraft conflict management, priority-based coordination and real-time visualisation in one browser-based simulation.

5.5 User Interface

The interface is a single page with an avionics-style look: a radar-style canvas showing the waypoint graph, aircraft, routes and predicted conflicts; a heads-up display (HUD) and per-aircraft panels; and simulation controls for starting and configuring scenarios.

6 Testing and Results

Testing was scenario-based. Four scenarios (A–D) exercise the main decision paths: right-of-way and A* bypass, multi-aircraft clustering, the holding fallback, and deterministic tie-breaking. Only results that were actually supplied are reported as observed; for the others the expected behaviour is stated and the observed outcome is to be recorded from a live run.

6.1 Test Cases

Case A: Head-on conflict. Two aircraft fly reciprocal routes. The aircraft with lower fuel gets the higher priority score P; the other yields and runs A* with penalties. *Expected:* the bypass keeps predicted separation above the 34 NM threshold. *Observed:* a minimum separation of 41.2 NM against the configured 34 NM (prototype result supplied by the team; not re-measured for this report).

Case B: Three-aircraft convergence. Three aircraft approach one common waypoint. Union-Find groups them and the priority order sets sequential route commitments. *Expected:* consistent yielding with no circular wait. *Observed:* all three aircraft reached their destinations without the circular-yielding deadlock (supplied prototype result; separation values to be recorded from a live run).

Case C: Holding-pattern contingency. All alternatives are blocked, or the detour exceeds the configured 35 NM of extra distance. The aircraft enters a simulated holding orbit and resumes routing when the modelled conflict clears. The simulation uses 210 kt, 25% of cruise fuel burn and the orbit $p_{\text{hold}}(t) = [x_{\text{fix}} + 16\cos\omega t,\ y_{\text{fix}} + 16\sin\omega t]$; these are simulation values only, not aviation guidance. *Observed:* no measurement supplied; to be demonstrated live.

Case D: Deterministic tie-breaking. Two aircraft have equal or near-equal scores, $|P_i - P_j| \leq \varepsilon = 0.6$, so the aircraft ID decides who yields. *Expected:* the same outcome on every run and no mutual yielding. *Observed:* no result supplied; to be run live and recorded.

Case	Scenario	Expected	Observed
A	Head-on conflict	Separation $>$ 34 NM after A* bypass	41.2 NM minimum separation (supplied)
B	Three-aircraft convergence	No circular wait	All reached destinations, no deadlock (supplied)
C	Holding contingency	Hold, then resume	Demonstrate live
D	Equal-priority tie	Same outcome every run	Record from live run

Table 9: Summary of test cases.

6.2 Live Demonstration Procedure

1. Start two aircraft on intersecting routes.
2. Show local sensing and conflict prediction.
3. Display priority negotiation.
4. Observe A* rerouting.
5. Introduce three-aircraft convergence.
6. Demonstrate holding or ID tie-breaking.
7. Check separation and destination completion.

6.3 Simulation Screenshots

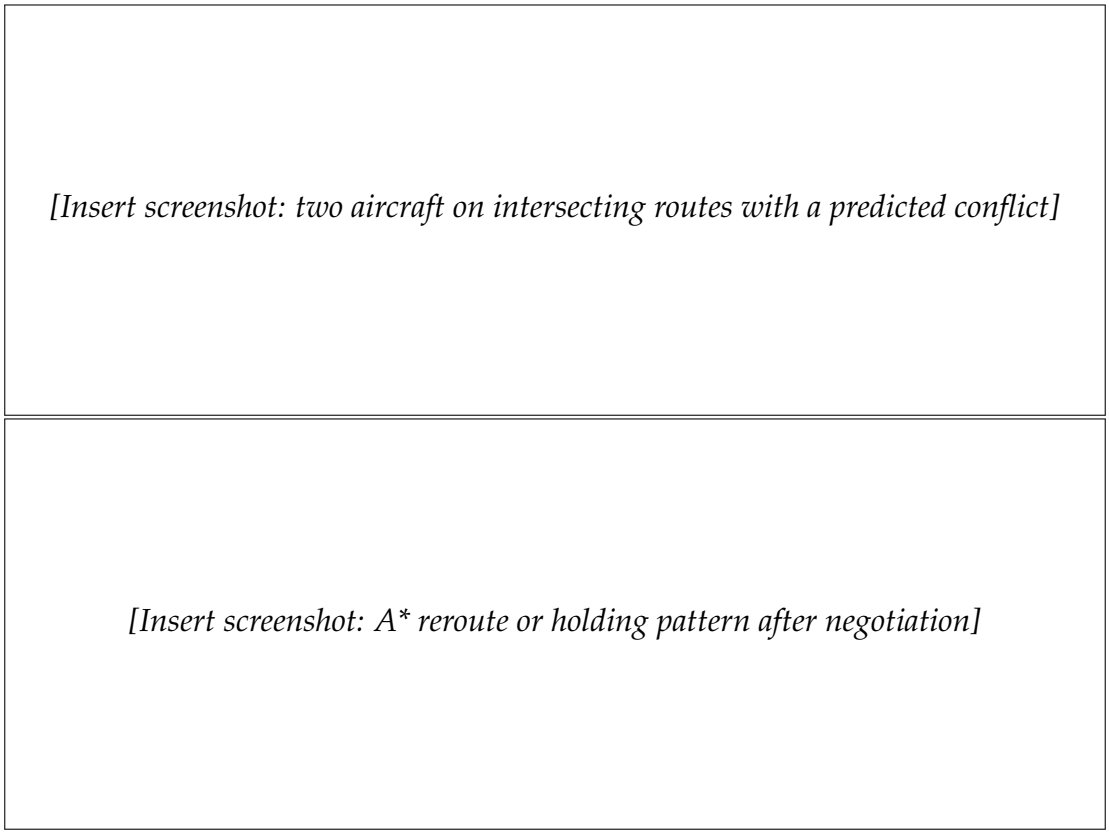


Figure 7: Simulation screenshots (to be inserted from the live demo).

6.4 Limitations

- Simplified 2D airspace and simulated sensing and aircraft dynamics.
- Thresholds and penalties are configured values, not derived from aviation standards.
- Limited large-scale testing evidence.
- No aviation certification; neither A* nor a weighted cost alone guarantees safety.

7 Conclusion and Future Work

SAFE SAFAR demonstrates the PEAS formulation, a multi-agent, partially observable environment, A* with a Euclidean heuristic, dynamic penalties with trajectory prediction, priority negotiation with deterministic tie-breaking, Union-Find clustering, and a holding-pattern fallback. Local perception and negotiation decide who yields, A* with dynamic penalties finds the alternative route, and holding covers cases with no acceptable detour.

Future improvements include 3D airspace with altitude separation, realistic dynamics with wind and fuel burn, standardised traffic-data interfaces, larger-scale stress tests,

and systematic safety validation.